

Регламент учебного проекта

1. Формат проекта

В течение курса студент или команда разрабатывает единое веб-приложение, последовательно дополняя его по мере изучения Go, баз данных, клиентской разработки, инфраструктуры, тестирования и безопасности.

Проект выполняется индивидуально или командой до трёх человек. До начала разработки необходимо согласовать с преподавателем тему, состав команды и зоны ответственности. Проект должен решать прикладную задачу и включать серверную часть на Go, реляционную базу данных, пользовательский web-интерфейс и воспроизводимое окружение запуска. Другой язык серверной части и использование клиента, созданного в рамках другого курса, допускаются только по предварительному согласованию.

Все исходные материалы, документы и принятые решения должны относиться к одному проекту и соответствовать фактической реализации.

Минимальный объём реализации

Процент готовности определяется по доле реализованных и успешно проверенных функциональных требований согласованного ТЗ.

Состав команды	Минимальный объём реализации
1 человек	Не менее 60% требований ТЗ либо 100% серверной части, клиент может быть упрощён или не разрабатываться
2 человека	Не менее 60% требований ТЗ, обязательны серверная и клиентская части
3 человека	100% функциональных требований ТЗ, развёртывание оценивается отдельно и не входит в процент готовности

Для команд из двух и трёх человек в ТЗ указывается зона ответственности каждого участника.

2. Техническое задание

ТЗ должно быть подготовлено и согласовано в течение первых 3-х недель семестра и содержать:

- назначение системы, целевых пользователей и их роли;
- функциональные и нефункциональные требования;
- состав серверной и клиентской частей, основные способы их взаимодействия или перечень API;
- требования к безопасности и ограничения проекта;
- распределение задач между участниками и этапы реализации;
- два измеримых технико-экономических показателя.

Изменение предметной области, существенных требований или технологического стека после согласования допускается только с разрешения преподавателя и фиксируется в новой версии ТЗ либо отдельном перечне изменений.

Для каждого участника определяются реализуемые функции, связанные с ними серверные компоненты или методы API, клиентские сценарии, структуры данных, критерии завершённости и способы проверки.

3. Обязательные требования к реализации

В проекте должны быть обеспечены:

- регистрация пользователей;
- аутентификация и авторизация с использованием JWT;
- ролевое разграничение доступа с обязательной проверкой прав на стороне сервера;
- защита аутентификации от перебора паролей;
- хеширование паролей с индивидуальной солью и секретным перцем без передачи и хранения паролей в открытом виде;
- корректная обработка штатных, ошибочных, граничных и запрещённых сценариев;
- создание схемы базы данных миграциями и воспроизводимая загрузка демонстрационных данных;
- доступность обязательных сценариев через пользовательский интерфейс и документированный API;
- сборка и запуск из чистой копии репозитория по инструкции без исправления исходного кода;
- развёртывание на выделенном сервере, если преподавателем не согласовано исключение.

Помимо перечисленного команда выбирает не менее одного дополнительного направления безопасности:

- шифрование хранимых данных;
- HTTPS и управление сертификатами, в том числе с собственной инфраструктурой сертификации;
- просмотр, завершение и отзыв пользовательских сессий;
- модель угроз разрабатываемой системы.

Выбранное направление фиксируется в ТЗ и отражается в реализации или проектной документации.

4. Репозиторий и проектные материалы

Проект ведётся в одном Git-репозитории с понятной структурой. В нём должны быть:

- README с назначением проекта, требованиями к окружению и командами запуска, сборки и тестирования;
- задачи или issue для крупных этапов и дефектов;
- отдельные ветки и содержательные pull request для существенных изменений;
- небольшие осмысленные коммиты, позволяющие определить вклад участников;
- защищённая основная ветка либо обязательная проверка изменений другим участником;
- зафиксированные версии основных зависимостей;
- исходный код серверной и клиентской частей;
- миграции, автоматизированные тесты и инструкция по их запуску;
- Dockerfile, конфигурация Docker Compose и файлы CI/CD.

Секреты, рабочие пароли, токены, приватные ключи и сгенерированные артефакты в репозитории хранить запрещено.

CI должен выполнять проверку форматирования, статический анализ, тестирование и сборку. Критические сценарии покрываются тестами, а известные критические дефекты устраняются до итоговой защиты.

К итоговой защите также представляются техническое задание, пояснительная записка, ПМИ, ИПП, демонстрационный набор данных без персональной и конфиденциальной информации и финальная презентация.

5. Этапы и контрольные мероприятия

Проект выполняется в три этапа:

1. **Первый этап:** базовая архитектура, основные модели данных, регистрация, аутентификация и начальный сквозной пользовательский сценарий.
2. **Второй этап:** основная бизнес-логика, взаимодействие серверной и клиентской частей, обработка ошибок и проверка прав доступа.
3. **Третий этап:** оставшаяся функциональность, исправление дефектов, документация и итоговые испытания, результаты которых отражаются в ИПП и ПМИ.

Срок	Контрольное мероприятие
Неделя 3	Согласование ТЗ, плана, распределения ответственности, этапов и дополнительного направления безопасности
Неделя 8	Тестирование по первой половине курса и индивидуальная защита первого этапа
Неделя 12	Индивидуальная защита второго этапа
Неделя 16	Тестирование по второй половине курса
Итоговая защита	Проверка третьего этапа, проектных документов и готового приложения

На защите студент демонстрирует свою часть проекта, показывает соответствующий код, объясняет технические решения, отвечает на вопросы по архитектуре, API, данным и безопасности и по требованию преподавателя вносит небольшое изменение. Каждый участник должен понимать общую архитектуру и взаимодействие компонентов, а не только собственный код.

К итоговой защите допускается проект, который соответствует согласованному ТЗ и обязательным требованиям настоящего регламента, воспроизводимо запускается, не содержит критических дефектов и сопровождается актуальными документами.

6. Академическая добросовестность

Запрещаются копирование чужого проекта или существенных фрагментов без указания источника и понимания решения, передача своей работы для копирования, подмена личного вклада чужими коммитами, а также использование языковых моделей и кодогенерации в оцениваемой части проекта без отдельного разрешения.

ИИ допускается для генерации тестовых данных, работы с формулировками и разбора документации, если преподаватель не установил иное. Такое использование необходимо раскрыть. Студент обязан объяснить каждое принятое решение.

Оценивание учитывает полноту выполнения ТЗ, работоспособность и воспроизводимость проекта, качество реализации и документации, соблюдение требований безопасности, результаты контрольных мероприятий и подтверждённый личный вклад. Неспособность объяснить собственный код, архитектуру, запросы к базе данных, инфраструктуру или меры защиты означает невыполнение соответствующего пункта. В этом случае защита может быть остановлена, а работа направлена на повторную сдачу, вплоть до смены темы проекта.